VIT-AP UNIVERSITY · SCHOOL OF COMPUTER SCIENCE & ENGINEERING

PROJECT-BASED COMPONENT (PBC) CAPSTONE PRESENTATION

30-Minute Team Presentation Script & Evaluator Defense Guide

Project Title: Hybrid Cloud and Distributed Ledger E-Voting System

Subtitle: A Web2.5 Architecture for Secure, Gasless, and Auditable Elections

Guide: Dr. Nagendra Panini Challa, Associate Professor, SCOPE | Academic Year 2025–2026

1. 30-Minute Time Budget & Role Division

Speaker	Student Details	Assigned Scope / Slides	Time Allotted
Speaker 1	Student-1 (22BCE8319 / 24MIC7027)	Slides 1 to 4: The Foundation, Failure Modes & The Web2.5 Vision	00:00 – 07:00 (7 min)
Speaker 2	Student-2 (22BEC8319 / 24MIC7307)	Slides 5 to 7: Hedera vs. Ethereum, 4-Tier Architecture & Ballot Lifecycle	07:00 – 14:00 (7 min)
Speaker 3	Student-3 (22BCE8506 / 24MIC7302)	Slides 8 to 10: Concurrency Lock, Fail-Safe Rollback, Auditability & STRIDE	14:00 – 21:00 (7 min)
Speaker 4	Student-4 (22BCE8947 / 24MIC7348)	Live Implementation: Codebase Deep Dive, Frontend UI & MongoDB Walkthrough	21:00 – 28:00 (7 min)
Panel & Team	All 4 Members	Evaluator Question & Answer Defense	28:00 – 30:00+ (2+ min)

2. Speaker 1: The Foundation, Failure Modes & The Web2.5 Vision

Presenter: Student-1 (Roll: 22BCE8319 / 24MIC7027) | Time: 00:00 – 07:00 (7 Minutes)

Speaker 1: Student-1 (22BCE8319 / 24MIC7027)

Slide 1: Title Slide & Web2.5 Value Proposition

[Stage Cue: Stand upright, face the evaluation panel with confidence, make eye contact, and speak in a clear, measured pace.]

"Respected panel of evaluators, project guide Dr. Nagendra Panini Challa, and fellow peers: Good morning."

"Today, our team is presenting our capstone project: 'Hybrid Cloud and Distributed Ledger E-Voting System: A Web2.5 Architecture for Secure, Gasless, and Auditable Elections.'"

"Democratic and institutional decision-making has always rested upon a singular foundation: trust. But trust cannot simply be promised by an institution; it must be mathematically and architecturally guaranteed."

"Our project introduces a Web2.5 architecture that combines the responsive user experience of traditional cloud web applications with the tamper-proof finality of the Hedera Hashgraph distributed ledger. Operating with an Asynchronous Byzantine Fault Tolerant consensus engine, our platform achieves over 10,000 transactions per second at a fixed micro-cost of $0.0001 per vote, delivering true End-to-End Verifiability without forcing voters to manage crypto wallets or pay gas fees."

Speaker 1: Student-1 (22BCE8319 / 24MIC7027)

Slide 2: The Evolution and Importance of Electoral Integrity

[Stage Cue: Transition slide; speak with gravitas regarding systemic vulnerabilities in centralized infrastructure.]

"To understand why this architecture is necessary, let us look at Slide 2: The Evolution of Electoral Integrity."

"Historically, paper ballots provided physical custody, but suffered from slow counting, ballot box stuffing, and logistical fragility. The global shift toward Electronic Voting Machines (EVMs) and digital centralized web portals solved tabulation speed—votes can now be counted in seconds."

"However, digital convenience introduced a systemic modern flaw: Extreme Centralization. When an election relies entirely on centralized servers and SQL or NoSQL databases, the entire foundation of institutional trust rests on the assumption that database administrators, cloud hosting providers, and privileged insiders will never behave maliciously or make mistakes."

"Distributed Ledger Technology (DLT) offers the definitive remedy: organizing data into an immutable, cryptographically chained sequence across globally decentralized nodes. But until now, blockchain voting has remained largely theoretical due to severe practical friction."

Speaker 1: Student-1 (22BCE8319 / 24MIC7027)

Slide 3: Centralized Vulnerabilities vs. Pure Web3 Usability Friction

[Stage Cue: Use contrasting hand gestures when comparing the two failure modes side-by-side.]

"On Slide 3, we analyze why both existing paradigms fail independently when applied to real-world institutional elections."

"Look at the left column: Centralized System Failures. First, Single Point of Database Authority: A privileged root DBA can run raw update queries to mutate vote counts, delete audit logs, or insert ghost ballots with zero external tamper-evidence. Second, Race Conditions & Double-Voting: Traditional check-then-write code allows rapid, concurrent API requests to slip past validation, recording duplicate ballots before the flag updates. Third, Absence of Independent Auditing: Voters receive an unprovable 'Vote Recorded' screen and are forced to blindly trust the central administration."

"Now, look at the right column: Pure Web3 dApp Friction. To solve centralized tampering, previous academic papers suggested pure decentralized dApps on Ethereum. But in practice, this introduces catastrophic usability barriers: First, Wallet Complexity: Every student would need to install browser extensions like MetaMask, write down 12-word seed phrases, and manage cryptographic private keys. Second, Economic Friction & Gas Volatility: Voters must purchase volatile cryptocurrencies. On Ethereum, gas spikes range from $5 to $50 per transaction, making an election cost thousands of dollars."

"In short: Centralized systems lack trust. Pure Web3 systems lack usability. Our goal is to resolve this exact tension."

Speaker 1: Student-1 (22BCE8319 / 24MIC7027)

Slide 4: Bridging Web2 Usability with Web3 Trust (The 4 Pillars)

[Stage Cue: Enumerate the four pillars clearly, emphasizing the gasless operator-relayer pattern.]

"This brings us to Slide 4: Bridging Web2 Usability with Web3 Trust. Our solution is founded on four core architectural pillars:"

"1. Gasless Operator-Relayer Pattern: We decouple the student's browser from blockchain transactions. The voter never signs a Web3 transaction directly. Instead, our backend relayer signs the ballot using an institutional Operator Key and pays the fraction-of-a-cent fee on their behalf."

"2. Zero Cost & Zero Friction: A student logs in through a familiar, clean web interface using their university credentials. No crypto wallets, no seed phrases, and zero transaction fees for the voter."

"3. Strict Double-Voting Prevention: We enforce atomic state transitions at the database tier using conditional document-level write locks, instantly terminating concurrent attempts in zero milliseconds."

"4. End-to-End Verifiability: Anonymized vote payloads are anchored immutably to the Hedera Consensus Service. Any auditor or student can verify the election tallies directly from independent mirror nodes without trusting our server."

"To explain how this consensus engine functions and how our four-tier architecture is organized, I now hand over to Speaker 2."

3. Speaker 2: The Consensus Engine, Architecture & Ballot Lifecycle

Presenter: Student-2 (Roll: 22BEC8319 / 24MIC7307) | Time: 07:00 – 14:00 (7 Minutes)

Speaker 2: Student-2 (22BEC8319 / 24MIC7307)

Slide 5: Bypassing Ethereum's Limitations for Enterprise Scale

[Stage Cue: Step forward, acknowledge Speaker 1, and focus on the technical metrics and distributed systems proofs.]

"Thank you, Speaker 1. Good morning, evaluators."

"Moving to Slide 5, our selection of Hedera Hashgraph over Ethereum Layer-1 is not a cosmetic choice; it is an empirical engineering decision based on three distributed systems requirements:"

"1. Throughput at Scale: Ethereum L1 handles only 15 to 30 transactions per second globally. During a campus election where thousands of students vote concurrently between classes, Ethereum suffers catastrophic mempool congestion. Hedera delivers over 10,000 TPS—a 333-times throughput advantage."

"2. Fixed Fiat-Pegged Economics: Ethereum fees fluctuate wildly based on DeFi network demand. Hedera’s transaction fees are pegged directly to the US Dollar by the Hedera Governing Council and fixed at exactly $0.0001 USD per message. Conducting an election for 10,000 students costs exactly $1.00 USD total."

"3. Asynchronous Byzantine Fault Tolerance (aBFT): Hashgraph achieves the gold standard in distributed security. It is mathematically proven to tolerate up to one-third malicious nodes (n >= 3f + 1) without partitioning or soft forks."

"4. Fair Ordering via Virtual Voting: Unlike traditional blockchains where miners or validators can reorder transactions or front-run ballots, Hedera uses a Gossip-about-Gossip protocol. Nodes calculate consensus ordering in local memory via virtual voting, assigning each vote a tamper-proof median consensus timestamp."

Speaker 2: Student-2 (22BEC8319 / 24MIC7307)

Slide 6: Decoupled for Speed, Security, and Scalability (The 4 Tiers)

[Stage Cue: Trace the pipeline from Tier 1 to Tier 4 using the slide diagram.]

"On Slide 6, you see how our system decouples concerns across Four Discrete Tiers, ensuring that a failure in one tier never cascades into another:"

"Tier 1: The Presentation Tier (React 18 & Vite): A responsive, accessible Single Page Application. It renders candidate manifestos, interactive comparison matrices, and the voter booth, communicating exclusively via authenticated REST API calls."

"Tier 2: The Application Relayer Tier (Node.js & Express): This is our secure gateway. It validates stateless JWTs, enforces Role-Based Access Control, strips all Personally Identifiable Information from the ballot, and securely holds the institutional Operator Key."

"Tier 3: The State Caching Tier (MongoDB): A high-speed document store running the WiredTiger engine. It manages student enrollment records, hashed credentials, and acts as our concurrency barrier via atomic conditional updates."

"Tier 4: The Distributed Consensus Tier (Hedera HCS): The global decentralized trust anchor. Operating on public Topic ID 0.0.10697996, it assigns an immutable consensus timestamp, running SHA-384 hash, and sequence number to every vote cast."

Speaker 2: Student-2 (22BEC8319 / 24MIC7307)

Slide 7: Step-by-Step Data Flow of a Cast Ballot

[Stage Cue: Walk through stages 01 to 06 with deliberate precision.]

"Now, look at Slide 7: The Step-by-Step Data Flow of a Cast Ballot. Every single vote traverses six auditable stages:"

"Stage 01 — Authentication: The student logs into the portal. The backend verifies their password against a bcrypt salt hash (cost factor 10) and issues a stateless 24-hour signed JWT."

"Stage 02 — Selection: The voter reviews candidate commitments on the React frontend and clicks 'Submit Vote' for their chosen candidate ID."

"Stage 03 — Atomic State Claim: Before any network call leaves our server, MongoDB executes an atomic findOneAndUpdate querying { voterId, hasVoted: false }. This single operation acts as an impenetrable gatekeeper against double-voting."

"Stage 04 — Payload Construction & PII Stripping: The relayer strips the student's name, email, roll number, and IP address. The payload contains only: { candidateId, timestamp }. Ballot secrecy is preserved."

"Stage 05 — Hashgraph Broadcast: The Node.js gateway instantiates a TopicMessageSubmitTransaction, signs it using the Operator Key, and dispatches it over gRPC TLS to Hedera Topic 0.0.10697996."

"Stage 06 — Receipt Return: Hedera orders the transaction and returns a consensus receipt. The server passes the cryptographic Transaction ID (formatted as AccountId-Seconds-Nanoseconds) back to the student's browser as permanent, verifiable proof."

"To explain how our system prevents distributed race conditions and allows anyone to audit results without trusting our database, I now pass the floor to Speaker 3."

4. Speaker 3: Concurrency, Independent Auditability & Security Architecture

Presenter: Student-3 (Roll: 22BCE8506 / 24MIC7302) | Time: 14:00 – 21:00 (7 Minutes)

Speaker 3: Student-3 (22BCE8506 / 24MIC7302)

Slide 8: Preventing Double-Voting and Handling Network Faults

[Stage Cue: Speak with technical authority on concurrency and distributed edge-case handling.]

"Thank you, Speaker 2. Respected evaluators, let us look at Slide 8, which addresses one of the most critical vulnerabilities in digital voting: Concurrency and Network Faults."

"In high-traffic elections, a malicious user or bot might fire 50 concurrent HTTP requests in the same millisecond. In a standard architecture, this creates a check-then-write race condition: all 50 threads check if hasVoted is false, all 50 threads see false, and all 50 cast a ballot."

"In our system, this is mathematically impossible. We utilize MongoDB WiredTiger's document-level locking via findOneAndUpdate({ voterId: req.voterId, hasVoted: false }, { $set: { hasVoted: true } }). Under document-level serialization, the very first thread acquires an exclusive write lock and mutates hasVoted to true. The remaining 49 concurrent threads evaluate the filter, find zero matching documents, evaluate to null, and are immediately rejected with an HTTP 403 Forbidden. Double-voting is eliminated at the database engine level."

"But what happens during a network fault? If MongoDB locks the record but university Wi-Fi drops before the Hedera transaction completes, our automated catch block intercepts the error and executes an atomic rollback: setting hasVoted: false. The student is never disenfranchised due to a transient network timeout."

"Furthermore, we handle Hedera SDK precheck retries (DUPLICATE_TRANSACTION), ensuring that temporary consensus delays do not trigger false-negative rollbacks. The result is zero double-votes and zero voter disenfranchisement."

Speaker 3: Student-3 (22BCE8506 / 24MIC7302)

Slide 9: Proving Election Integrity Without Trusting the Database

[Stage Cue: Emphasize with authority that MongoDB does NOT count the votes.]

"Moving to Slide 9, we present the most pivotal architectural breakthrough of our project: Proving Election Integrity Without Trusting the Database."

"In centralized voting systems, the database calculates the final tally. If the database is compromised, the election is stolen. In our architecture:"

"1. Decoupled Tallying: MongoDB manages voter eligibility only. Final tallies are never calculated from MongoDB. Even if an attacker gains root access to our MongoDB instance, altering records in the database does not change a single vote count."

"2. Direct Ledger Streaming: When someone views election results, our React frontend streams consensus messages directly from public Hedera Mirror Node REST APIs. It fetches the chronological transaction stream from Topic 0.0.10697996 and tallies votes client-side. The ledger is the sole source

of truth."

"3. Cryptographic Receipts: Every voter receives a receipt formatted as AccountId-Seconds-Nanoseconds (for example, 0.0.10697996@1718000000.123456789)."

"4. Independent Verification on HashScan: Any student can take their transaction ID and look it up on HashScan.io. They can independently prove that their ballot was assigned an official nanosecond consensus timestamp and chained into the global SHA-384 running hash—without revealing their identity."

Speaker 3: Student-3 (22BCE8506 / 24MIC7302)

Slide 10: Military-Grade Controls at Every Layer (STRIDE Evaluation)

[Stage Cue: Highlight the STRIDE matrix and key quantitative engineering metrics.]

"Finally, on Slide 10, we summarize our security posture using the industry-standard STRIDE Threat Modeling Framework:"

"Spoofing: Mitigated by salted bcrypt password hashing (cost factor 10) and cryptographically signed stateless JWTs with strict 24-hour expiration windows."

"Tampering: Zero risk on the ledger. Once a vote enters Hedera HCS, altering it would require breaking SHA-384 hash chaining and corrupting a global decentralized governing council."

"Repudiation: Completely eliminated. The cryptographic transaction receipt serves as non-repudiable mathematical proof of inclusion."

"Information Disclosure: Voter PII is strictly quarantined off-chain in private institutional MongoDB storage. The public ledger contains zero student names or IDs."

"Denial of Service: Handled by Node.js non-blocking asynchronous event loops and MongoDB unique sparse indexes, preventing thread starvation during voting surges."

"Elevation of Privilege: Enforced via requireRole('Admins') middleware, isolating administrative election toggles from voter permissions."

"To show you exactly how these architectural invariants are implemented in production code, how the interface functions, and to inspect our live database, I now invite Speaker 4 for the live demonstration."

5. Speaker 4: Code Overview, Frontend Interface & Live MongoDB Walkthrough

Presenter: Student-4 (Roll: 22BCE8947 / 24MIC7348) | Time: 21:00 – 28:00 (7 Minutes)

Speaker 4: Student-4 (22BCE8947 / 24MIC7348)

Live Code Deep Dive: Backend Core Architecture in server.js (Lines 290–355)

[Stage Cue: Transition to laptop screen; open VS Code showing backend/server.js lines 290–355.]

"Thank you, Speaker 3. Respected evaluators, I will now walk you through the codebase, the user interface we developed, and our live MongoDB database."

"As you can see in our project explorer, the codebase is structured as a decoupled monorepo: backend/ running Node.js and the Hedera SDK, and frontend/ powered by React 18 and Vite."

"Let us look at backend/server.js: On lines 62 to 76, we initialize the Hedera Client using @hashgraph/sdk. We configure our operator account credentials and establish an authenticated gRPC channel to the Hedera Testnet. On lines 80 to 100, we define our authenticateToken and requireRole middleware, validating the JWT claims before any request can touch election logic."

"Now, observe lines 290 to 355: this is our core ballot endpoint, app.post('/castVote'). Notice the atomic claim on lines 307 to 311: const voter = await Voter.findOneAndUpdate({ voterId: voterId, hasVoted: false }, { hasVoted: true, votedAt: new Date() }, { returnDocument: 'after' });. This is the exact code implementing Speaker 3's concurrency explanation. It acquires an exclusive document-level write lock. If a voter double-clicks or replays requests, the second thread receives null and is halted on line 319 with an HTTP 403 Forbidden."

"Next, on lines 324 to 336, we construct the anonymized payload containing only { candidateId, timestamp }. We instantiate a TopicMessageSubmitTransaction targeting Topic 0.0.10697996, execute it on Hedera, and obtain the receipt."

"Finally, look at our fail-safe catch block on lines 337 to 345: if the Hedera network fails or times out, the server immediately triggers a rollback: await Voter.findOneAndUpdate({ voterId: voterId }, { hasVoted: false, votedAt: null });. This guarantees that network interruptions never cause voter disenfranchisement."

Speaker 4: Student-4 (22BCE8947 / 24MIC7348)

Frontend Architecture: React 18 User Interface & Minimalist Off-White Aesthetic

[Stage Cue: Switch window to Chrome running at http://localhost:5173.]

"Now, let us examine the user interface in frontend/."

"In accordance with institutional design standards, we engineered a minimalist, warm studio off-white aesthetic (#faf9f5 canvas, crisp 1px borders, deep charcoal typography, and subtle pine green accents), completely replacing noisy gradients with clear information architecture."

"We added three primary interface modules:"

"1. The Manifesto Dashboard (ManifestoDashboard.jsx): Instead of presenting an opaque list of names, voters land on an educational civic portal. Students can review detailed candidate policy pillars, past incumbent completion scores, and linked university work-order evidence before entering the polling

booth."

"2. The Secure Voter Booth (VoterBooth.jsx): Inside the booth, candidates are rendered as interactive cards. When the voter selects a candidate and submits, an interactive tactile audio confirmation plays, the button enters an animated loading state, and upon consensus confirmation, the UI displays a cryptographic modal showing their Hedera Transaction ID."

"3. The Administrator Console (AdminConsole.jsx): Administrators can enroll eligible voters with bcrypt-hashed credentials, toggle the election active or inactive, and inspect live ledger tallies."

Speaker 4: Student-4 (22BCE8947 / 24MIC7348)

Live MongoDB Inspection: Real-Time Data Verification in MongoDB Compass

[Stage Cue: Switch window to MongoDB Compass showing voting_system database and voters collection.]

"Now, I will open MongoDB Compass so you can inspect the underlying data structures in real time."

"Look at the documents in our voters collection: Notice the password field for voter 22BCE8319: it contains an authentic 60-character bcrypt hash ($2b$10$...). Passwords are never stored in plaintext. Look at the role field: set to 'voter'. Notice hasVoted: currently set to false, and votedAt is null."

"When a vote is submitted, hasVoted transitions atomically to true, and votedAt records the timestamp."

"Most importantly, evaluators, look at what is NOT in this database document: There is no candidate choice field. The database records that student 22BCE8319 has participated, but it has no technical knowledge of which candidate they selected."

"The candidate choice exists exclusively inside the anonymized payload on Hedera Hashgraph Topic 0.0.10697996. Even a DBA with full root permissions cannot determine how an individual student voted."

Speaker 4: Student-4 (22BCE8947 / 24MIC7348)

Project Synthesis: Final Contributions & Formal Panel Hand-Off

[Stage Cue: Return to the title slide on projector; stand alongside all 3 team members.]

"To conclude our presentation:"

"1. We resolved the fundamental tension between centralized vulnerability and Web3 friction through our Gasless Web2.5 Operator-Relayer model."

"2. We proved that MongoDB WiredTiger document locking eliminates race conditions and double-voting under concurrent load."

"3. We decoupled tallying from the database, streaming live election results directly from Hedera Mirror Nodes."

"4. We delivered a complete, production-ready system with candidate manifestos, audit verification, and end-to-end voter privacy."

"We thank our project guide, Dr. Nagendra Panini Challa, and the School of Computer Science and Engineering. We are now open for questions from the evaluation panel."

6. Evaluator Defense Cheat Sheet (Q&A Defense Guide)

Comprehensive answers prepared for the examination committee.

Q1: If your backend holds the Operator Key, isn't that a centralized point of failure?

Primary Defender: Speaker 2

Defensive Answer Strategy:

"No. The Operator Key only signs and pays for message submission; it cannot alter the consensus ordering, modify the running hash, or tamper with consensus timestamps. For production mainnet, we migrate the single operator key to a Hedera multi-signature threshold account requiring 3 of 5 faculty election commissioners to co-sign."

Q2: How do you mathematically guarantee that two votes aren't cast at the exact same millisecond?

Primary Defender: Speaker 3 / Speaker 4

Defensive Answer Strategy:

"Through MongoDB WiredTiger document-level locking. The filter condition { voterId, hasVoted: false } serializes write requests. The first thread locks the document and updates hasVoted: true. The second thread evaluates the filter, finds zero matching documents, returns null, and our code halts execution with HTTP 403 before any network call occurs."

Q3: Can a student prove they voted without revealing who they voted for?

Primary Defender: Speaker 1 / Speaker 3

Defensive Answer Strategy:

"Yes. When their vote succeeds, they receive a cryptographic Transaction ID receipt. They can search this ID on HashScan.io to prove that their ballot was included at an exact nanosecond timestamp. Because the payload contains only { candidateId, timestamp } without student PII, their participation is mathematically proven while ballot secrecy remains intact."

Q4: Why not use Polygon or Arbitrum rollups instead of Hedera?

Primary Defender: Speaker 2

Defensive Answer Strategy:

"Two reasons: First, gas fee predictability: Polygon gas still fluctuates based on network demand, while Hedera's fees are fixed in USD at $0.0001 per message. Second, finality time: Polygon rollups require 2 to 4 minutes for state commitment to Ethereum L1. Hedera achieves deterministic aBFT finality in 3.8 seconds with zero fork risk."

Q5: What happens if a student claims they voted, but the university administration claims they

didn't?

Primary Defender: Speaker 4

Defensive Answer Strategy:

"This is the Non-Repudiation guarantee of our STRIDE evaluation. When a vote succeeds, the voter's browser receives and saves a cryptographic Transaction ID. The student can search that transaction ID on hashscan.io. The public Hedera network independently proves whether that transaction was included in Topic 0.0.10697996, at what exact nanosecond, and with what sequence number. Neither party can dispute a verified on-chain consensus receipt."

7. Pre-Presentation Technical Checklist

Ensure all local environments and tabs are tested prior to stepping onto the stage:

1. Backend Service: node server.js running on port 5000 connected to MongoDB and Hedera Testnet.
2. Frontend Service: npm run dev running on port 5173 with browser opened to http://localhost:5173.
3. Browser Tab 1: Frontend Login & Voter Dashboard ready (VOTER001 credentials pre-filled).
4. Browser Tab 2: HashScan Explorer loaded (https://hashscan.io/testnet/topic/0.0.10697996).
5. Browser Tab 3: Mirror Node REST API endpoint pre-tested and ready.
6. MongoDB Compass: Open and connected to voting_system database on localhost:27017.
7. Backup Connectivity: Mobile hotspot ready in case campus Wi-Fi blocks Hedera gRPC port 50211.